

2. XSS

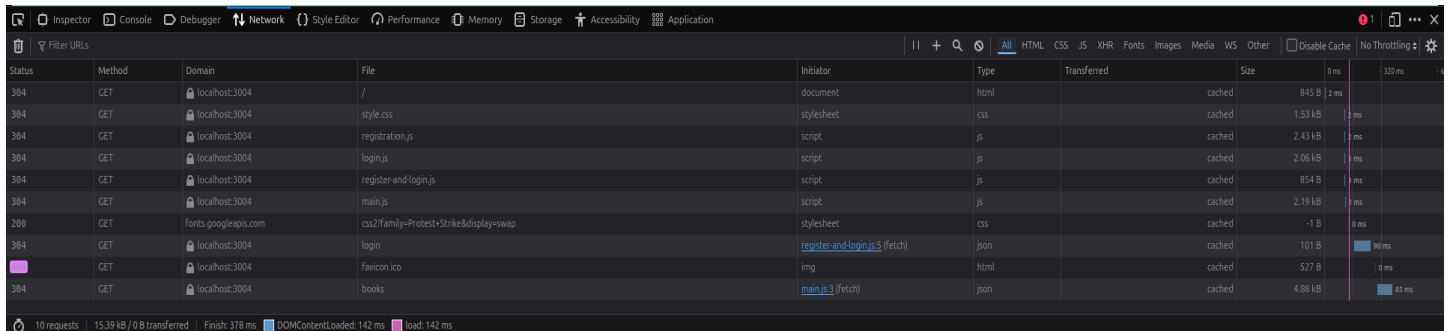
Challenge 1

Make an XSS attack as a regular registered user and that destroys for a superadmin so that they are directly redirected to <https://expressen.se> as soon as they have logged in

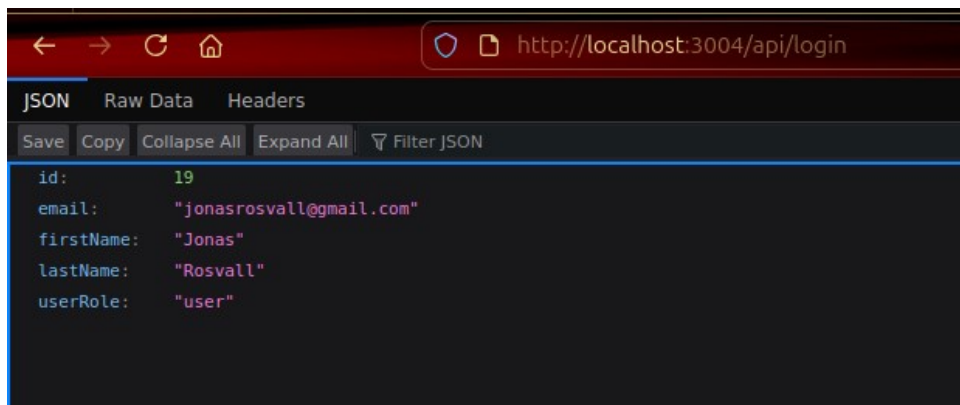
Preconditions

1. User registration form allows for user registration with name, last name and email (no verification)

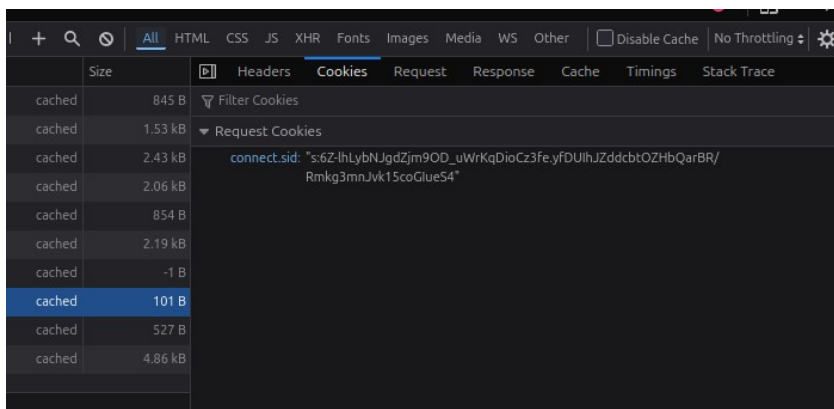
I register user Jonas Rosvall with jonasrosvall@gmail.com. User details can be accessed via `/api/login` as traced from the console



Status	Method	Domain	File	Initiator	Type	Transferred	Size	Time
304	GET	localhost:3004	/	document	html		845 B	0 ms
304	GET	localhost:3004	style.css	stylesheet	css		1.53 kB	2 ms
304	GET	localhost:3004	registration.js	script	js		2.43 kB	2 ms
304	GET	localhost:3004	login.js	script	js		2.06 kB	1 ms
304	GET	localhost:3004	register-and-login.js	script	js		854 B	1 ms
304	GET	localhost:3004	main.js	script	js		2.19 kB	1 ms
200	GET	fonts.googleapis.com	css?family=Protest+Strike&display=swap	stylesheet	css		-1 B	0 ms
304	GET	localhost:3004	login	register-and-login.js:5 (fetch)	json		101 B	90 ms
304	GET	localhost:3004	favicon.ico		img		527 B	0 ms
304	GET	localhost:3004	books	main.js:3 (fetch)	json		4.86 kB	83 ms



2. The login uses cookie-based authentication. To access this: Console → Network → any API (I chose `login`) → Cookies → Request Cookies



Size	Headers	Cookies	Request	Response	Cache	Timings	Stack Trace
cached	845 B	Filter Cookies					
cached	1.53 kB	Request Cookies					
cached	2.43 kB	connect.sid: "s:6Z-lhLybNUgdZjm9OD_uWrKqDioCz3fe.yfDUihJZddcbtOZHbQarBR/Rmkg3mnJvk15coGlue54"					
cached	2.06 kB						
cached	854 B						
cached	2.19 kB						
cached	-1 B						
cached	101 B						
cached	527 B						
cached	4.86 kB						

3. No implementation by the front of the “injectable” fields beyond user registration form. `/api/edit-my-user-info` as specified will be used to hack the website

Steps taken:

1. Register and login as a standard user Jonas Rosvall
 2. Enter console and identify API endpoints and the request cookie (connect.sid) that authenticates the login request. The reason why I'm using cookie is to access a piece of unpublished code from a 3rd party software from a logged in account.
 4. Enter the following details into the Thunder Client
- Method: PUT → <https://www.localhost:3004/api/edit-my-user-info>

Headers:

Accept: */*

User-Agent: Thunder Client (<https://www.thunderclient.com>)

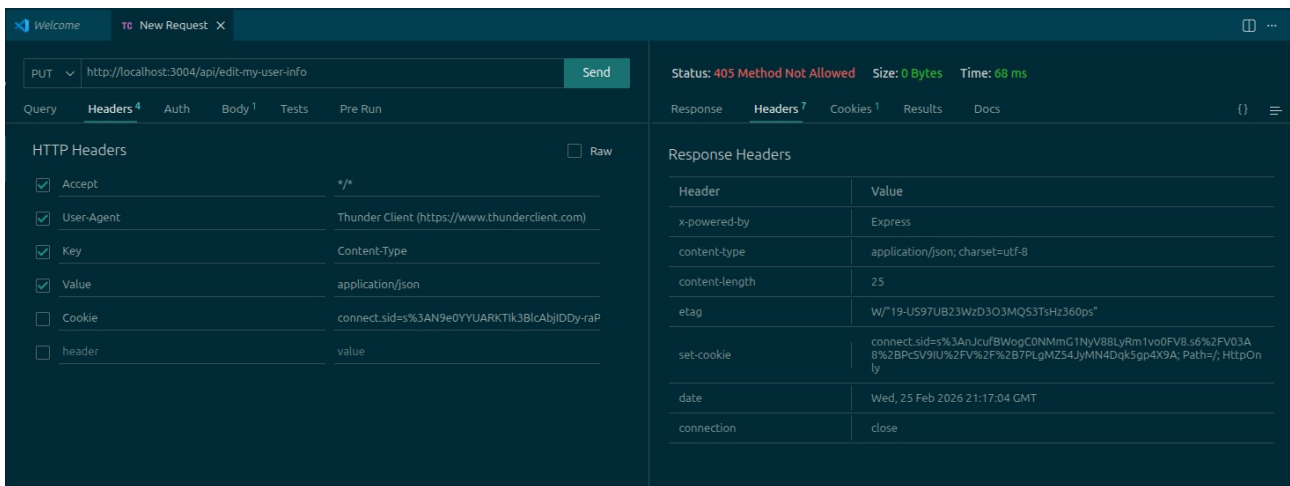
Key: Content-Type

Value: application/json

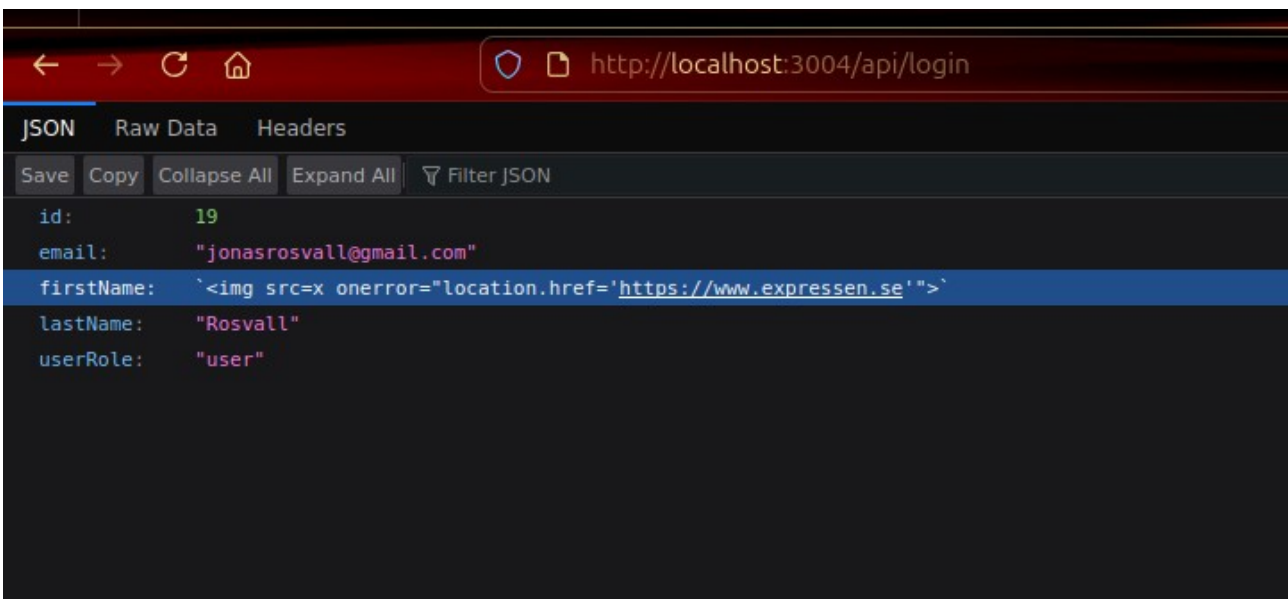
Cookie: connect.sid=s:6Z-lhLybNJgdZjm9OD_uWrKqDioCz3fe.yfDUlhJZddcbtOZHbQarBR/Rmkg3mnJvk15coGIueS4"

in Body → { "firstname": "" }

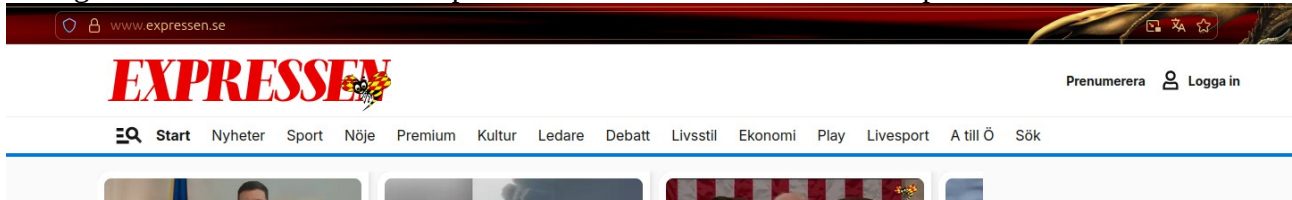
Screenshot shows the Thunder Client



and live change in API changing from the value of the first name to injected payload



5. Log in as Anna Larson with her password → automatic redirect → [expressen.se](https://www.expressen.se)



Result

The payload with the link to Expressen was stored in the database. When the admin logged in, their browser rendered the injected HTML, allowing JS execute payload when the admin logged in.

Why did the attack work? What was it in the code that made it possible?

Upon entering Console → Network → register-and-login.js there is a piece of code:

```
div.innerHTML = Logged in as ${loggedIn.firstName} ${loggedIn.lastName} <a  
href="/logout">Logout</a> and loggedIn = await (await fetch('/api/login')).json();
```

This means the input is controlled by the user. That same input is later stored in the database into the backend and then dynamically sourced by frontend at login. `innerHTML` tells the browser to treat whatever string it receives as HTML, not text — which becomes dangerous when that string contains user input. So even at the registration point, without using ThunderClient, if the `firstName` contains `<img src=x onerror="location.href='https://www.expressen.se'">` that payload is stored and sourced at login. So if the admin logs in, the payload executes automatically. Another piece of code that enables the hack is `start(loggedIn?.userRole)` that passes the user's role into the frontend logic, which then decides what data to render. If user has admin privileges the further implication is that they see a list with user names on their welcome screen (as stated in the workshop instructions). Since the admin sees all dynamic users details list right at the beginning, the payload would be executed immediately. *Immediately* being the operative word for the kind of payload that was injected. The mechanics of this specific code: `<img src=x onerror="location.href='https://www.expressen.se'">` is that an image is being prevented from loading from a corrupt source (`img src=x`) triggering the `onerror` handler, which executes JavaScript and redirects the page. The link to Expressen was loaded as response to an error in retrieving image source. The culprit of this attack is that `<img>` are loaded automatically, triggering instantaneous errors if unable to retrieve source therefore there is no way from stopping the payload from executing during the admin login.

Which category in the OWASP Top 10 belongs to the vulnerability?

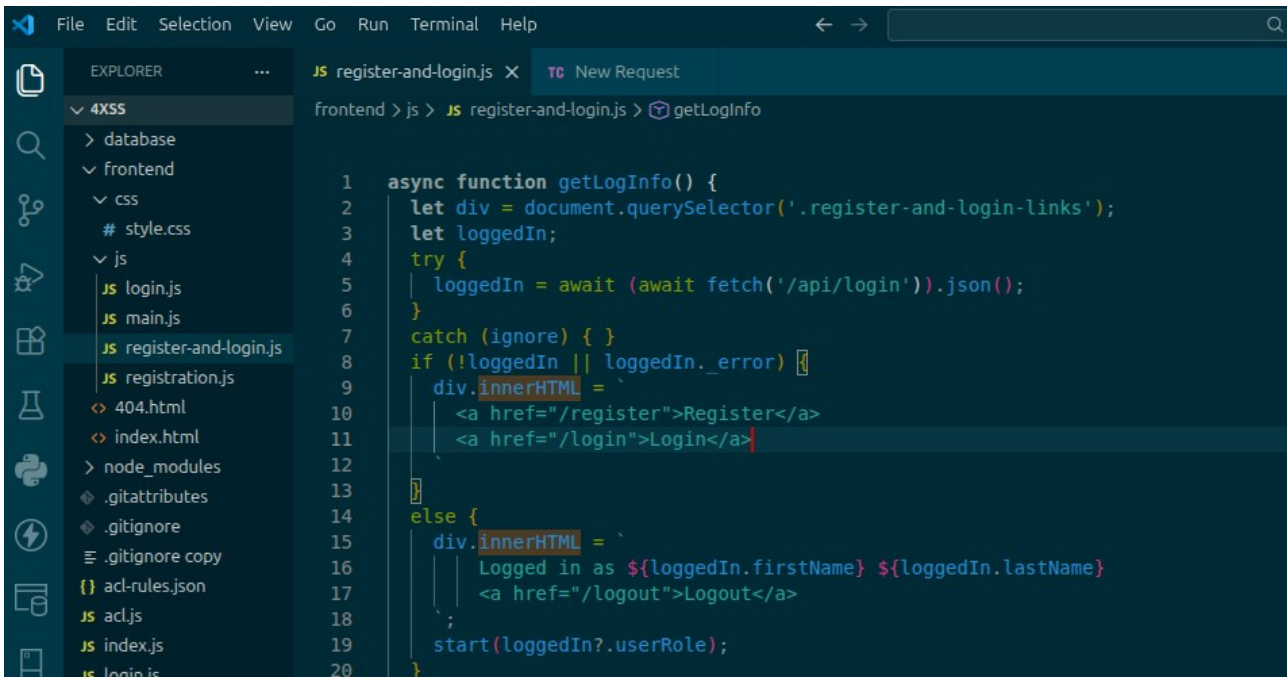
Stored Cross-Site Scripting (XSS) OWASP Top 10: A05 - Injection, because unverified input is interpreted as executable code in the browser.

How did you secure the code? Show what you changed and explain why it works.

The exploit occurs because user-controlled data (`firstName`) is inserted into the DOM using `innerHTML`. Since `innerHTML` parses and executes HTML and JS, any potentially damaging, payload stored in the database is executed in the browser when the page renders the content. Changing `innerHTML` to `textContent` in non-dynamic fields stops the injection from happening.

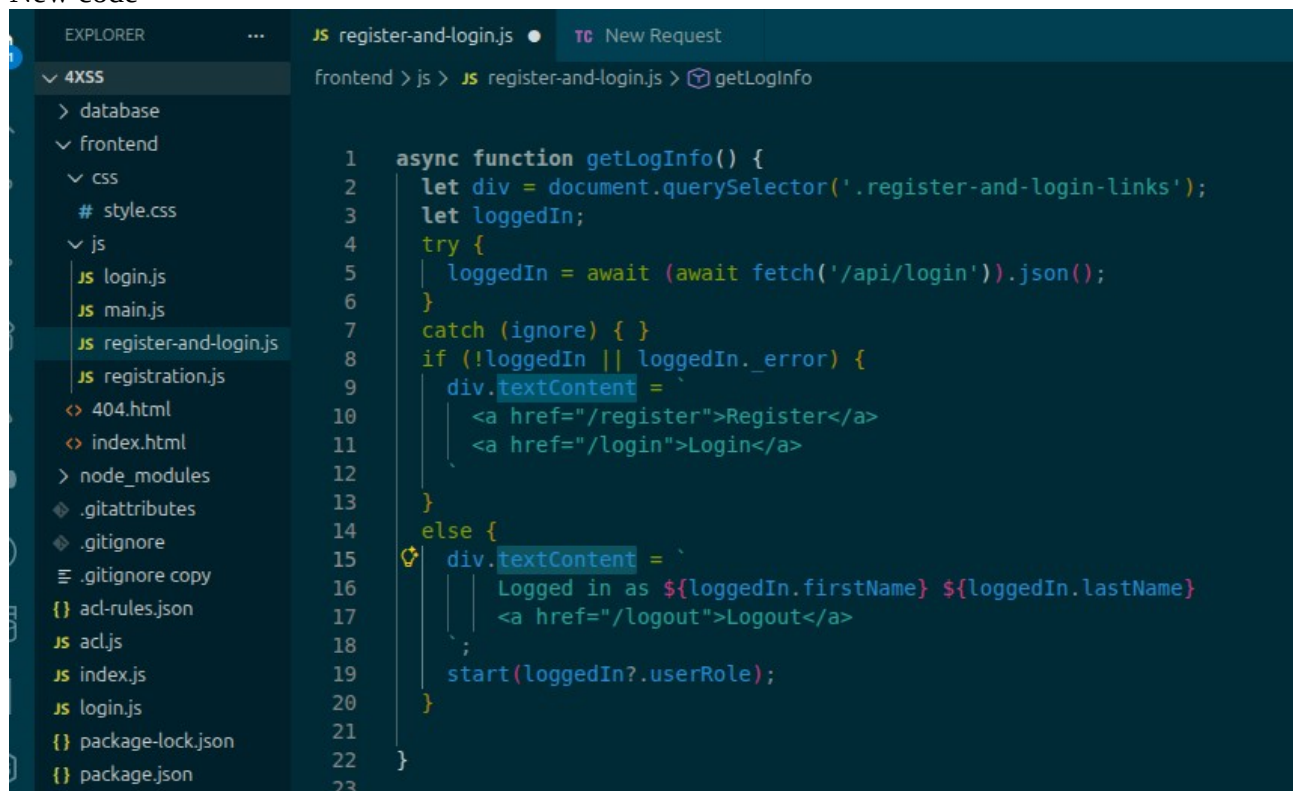
`textContent` makes it so that user input is seen as plaintext not HTML. This prevents the browser from parsing values like `<img>` or `<script>` and stops JS from execution.

Old code



```
1 async function getLogInfo() {
2   let div = document.querySelector('.register-and-login-links');
3   let loggedIn;
4   try {
5     loggedIn = await (await fetch('/api/login')).json();
6   }
7   catch (ignore) { }
8   if (!loggedIn || loggedIn._error) {
9     div.innerHTML = `
10      <a href="/register">Register</a>
11      <a href="/login">Login</a>
12    `;
13  }
14  else {
15    div.innerHTML = `
16      Logged in as ${loggedIn.firstName} ${loggedIn.lastName}
17      <a href="/logout">Logout</a>
18    `;
19    start(loggedIn?.userRole);
20  }
21 }
```

New code



```
1 async function getLogInfo() {
2   let div = document.querySelector('.register-and-login-links');
3   let loggedIn;
4   try {
5     loggedIn = await (await fetch('/api/login')).json();
6   }
7   catch (ignore) { }
8   if (!loggedIn || loggedIn._error) {
9     div.textContent = `
10      <a href="/register">Register</a>
11      <a href="/login">Login</a>
12    `;
13  }
14  else {
15    div.textContent = `
16      Logged in as ${loggedIn.firstName} ${loggedIn.lastName}
17      <a href="/logout">Logout</a>
18    `;
19    start(loggedIn?.userRole);
20  }
21 }
22
23 }
```

After replacing `innerHTML` with `textContent`, the injected payload is displayed as plaintext as seen in `firstName` value. No redirection to Expressen confirms that the attack no longer works.

Challenge 2

Try to make an attack that “raises” the user role on a regular user you registered to superadmin.

Steps taken:

1. Reload a clean copy of the download file into the Visual Code. Initiate the program, enter the browser, login as Jonas Rosvall.
2. Open Console → `fetch('/api/login').then(r => r.json()).then(console.log)` to check the current status of the user “Jonas Rosvall”. It should be stated as “user”
2. Extract cookie from the console (Console → Network → any API → Cookies → Request Cookies. Cookie in this scenario is central as it identifies currently logged user.
3. Initiate Thunder Client → Method PUT → `http://localhost:3004/api/edit-my-user-info` → Headers → Content-Type: application/json
Cookie: `connect.sid=s:lUJphrFoMjUEjNeeAPAQhJCPXTeb1ge5.3uWwFCAHgXhv88lFI/ndEOH6BtSejkN+dspLsTiqI0I`
Body → `{ "userRole": "superadmin" }` // reload
4. Check if successful: Console → `fetch('/api/login').then(r => r.json()).then(console.log)`. Jonas’s status is now logged in as “superadmin”

The before and after “privilege upgrade”: the screenshot shows Jonas initially with ordinary “user” privileges and then a “superadmin” after PUT request



```
▶ GET http://localhost:3004/favicon.ico
>> fetch('/api/login').then(r => r.json()).then(console.log)
← ▶ Promise { <state>: "pending" }
▶ Object { id: 19, email: "jonasrosvall@gmail.com", firstName: "Jonas", lastName: "Rosvall", userRole: "user" }
>> Cookie: connect.sid=s:lUJphrFoMjUEjNeeAPAQhJCPXTeb1ge5.3uWwFCAHgXhv88lFI/ndEOH6BtSejkN+dspLsTiqI0I
❗ Uncaught SyntaxError: unexpected token: ':' [Learn More]
>> fetch('/api/login').then(r => r.json()).then(console.log)
← ▶ Promise { <state>: "pending" }
▶ Object { id: 19, email: "jonasrosvall@gmail.com", firstName: "Jonas", lastName: "Rosvall", userRole: "superadmin" }
>>
```

Logic:

This attack feeds on bad authorisation logic of the API design. The *edit-my-user-info* API implies that the page intends to allow standard users to edit their own details. These details should be limited to what’s in control of the user like their own name, email, address if present etc. It should not give users the option to edit operational details such as userRole, admin status, permissions etc. The backend doesn’t validate or restrict what field the user can modify. It allows the user input to enter the database and the piece of code that says *start(loggedIn?.userRole)* switches the user roles based on what was previously registered in the backend so Jonas Rosvall can log in as superadmin upon the next refresh.

This example shows how the backend fails to enforce authorization rules and allows privilege escalation.

In terms of OWASP Top 10 this attack classifies as both **Broken Access Control (A01)** and **Insecure Design (A06)**

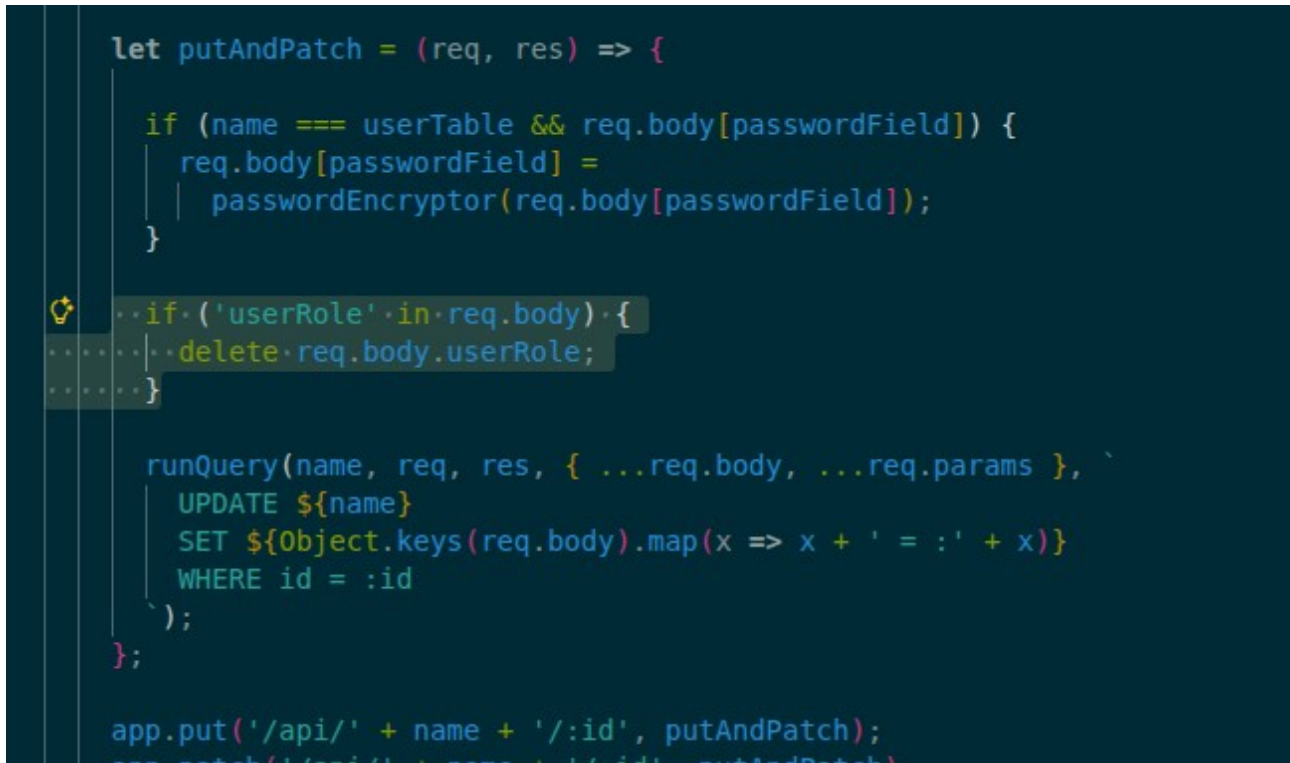
This attack doesn’t strictly classify as XSS but rather an exploit arisen by missing authorization checks in the API.

How did you secure the code? Show what you changed and explain why it works.

In Visual Code → Rest-apis.js → add the following line of code under *putAndPatch* section:

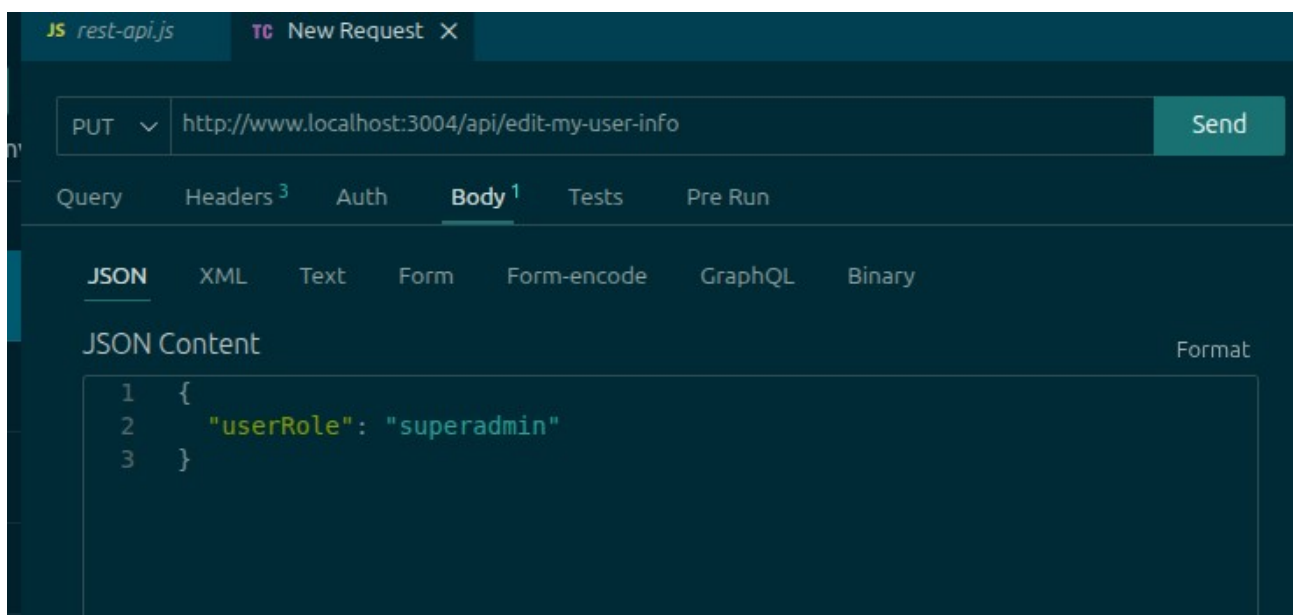
```
if ('userRole' in req.body) {  
  delete req.body.userRole; }
```

This specific line prevents the database from updating *userRole* that the user may want to try to attribute to himself, preventing ordinary users from augmenting privileges.



```
let putAndPatch = (req, res) => {  
  if (name === userTable && req.body[passwordField]) {  
    req.body[passwordField] =  
      passwordEncryptor(req.body[passwordField]);  
  }  
  if ('userRole' in req.body) {  
    delete req.body.userRole;  
  }  
  runQuery(name, req, res, { ...req.body, ...req.params }, `  
    UPDATE ${name}  
    SET ${Object.keys(req.body).map(x => x + ' = :' + x)}  
    WHERE id = :id  
  `);  
};  
  
app.put('/api/' + name + '/:id', putAndPatch);  
app.patch('/api/' + name + '/:id', putAndPatch);
```

The screenshot shows the same attempt to add admin privileges, made in Thunder Client as with the example before but this time after code implementation



```
PUT http://www.localhost:3004/api/edit-my-user-info  
  
JSON Content  
1 {  
2   "userRole": "superadmin"  
3 }
```

The changes show no change in user status for the user before and after code implementation (API restarted)

```
▶ GET http://localhost:3004/favicon.ico
>> (await fetch('/api/login')).json()
< ◀ Object { id: 19, email: "alisultan@gmail.com", firstName: "ali", lastName: "sultan", userRole: "user" }
  email: "alisultan@gmail.com"
  firstName: "ali"
  id: 19
  lastName: "sultan"
  userRole: "user"
  ▶ <prototype>: Object { ... }

>> (await fetch('/api/login')).json()
< ▶ Object { id: 19, email: "alisultan@gmail.com", firstName: "ali", lastName: "sultan", userRole: "user" }

>> await fetch('/api/login').json()
❗ Uncaught SyntaxError: unexpected token: ')' [Learn More]

>> (await fetch('/api/login')).json()
❗ Uncaught SyntaxError: unexpected token: ')' [Learn More]

>> (await fetch('/api/login')).json()
< ▶ Object { id: 19, email: "alisultan@gmail.com", firstName: "ali", lastName: "sultan", userRole: "user" }

>>
```

Risk analysis

The likelihood of XSS attack is high and impact moderate to high.

The attack requires no special privileges and can be executed by any user during registration. The payload is easy to construct and information about it is widely available online. Because the application stores the injected code in the database, the attack becomes persistent, meaning it will execute every time the affected content is presented.

Code-wise, the application is build on *innerHTML*, which widens the attack surface. Since no server input validation or output encoding is present, the exploit can be discovered and implemented fast. Even though the payload only redirects the user to another website, a real-world attacker could provide links to much more gruesome websites than Expressen, inject scripts, steal session cookies, act as logged-in users and tie their actions to user accounts, alter page content.

Because the attack is stored, it can affect multiple users automatically, including administrators if they view the affected content.

Defense Considerations

The application lacks multiple layers of protection. A more secure design would include:

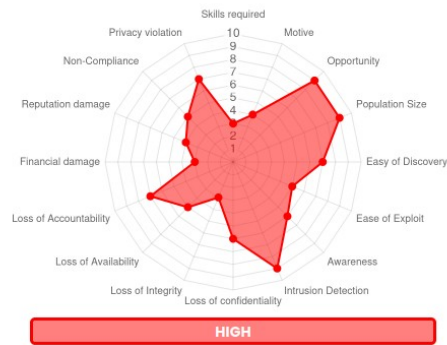
- input validation on the server
- output encoding before rendering user data
- avoiding unreliable DOM functions like *innerHTML*
- implementing Content Security Policy (CSP)

With this in mind even if one layer fails, the others would reduce the risk.

Limitations of the Current Fix

The implemented mitigation mainly replaces *innerHTML* with *textContent* in the API code. While this reduces the attack surface, it doesn't fully get rid of the exploit because previously stored input may still be in the database and because backend validation is still missing.

OWASP Risk Assessment Calculator



Threat Agent Factors

Skill level	Some technical skills (3)	▼
Motive	Possible reward (4)	▼
Opportunity	No access or resources required (9)	▼
Size	Anonymous Internet users (9)	▼

Technical Impact Factors

Loss of confidentiality	minimal critical data disclosed (6)	▼
Loss of integrity	Minimal seriously corrupt data (3)	▼
Loss of availability	Minimal primary services interrupted (5)	▼
Loss of accountability	Possibly traceable (7)	▼

Vulnerability Factors

Ease of discovery	Easy (7)	▼
Ease of exploit	Easy (5)	▼
Awareness	Obvious (6)	▼
Intrusion detection	Not logged (9)	▼

Business Impact Factors

Financial damage	Minor effect on annual profit (3)	▼
Reputation damage	Loss of major accounts (4)	▼
Non-compliance	Clear violation (5)	▼
Privacy violation	Thousands of people (7)	▼

Likelihood score

6.500 HIGH

Impact score

5.000 MEDIUM